

第 5 册 接口文件导航部署测试与维护

本册完整收录对应专题的原始内容，保留代码事实、现有限制与改进建议的区分。供开发维护和 AI 项目理解使用，不应直接发布为面向客户的售后政策。

本册专题：接口与文件导航；配置运行测试与维护

接口与文件导航

本篇用于从一个业务问题定位到真实文件，而不是要求读者背诵目录。以下路径以仓库根目录为基准。请求参数和响应字段的最终依据是对应路由、Schema 及运行中的 `/docs`；表格只总结接口目的，不取代完整 OpenAPI。

接口分组

`GET /health` 提供进程健康信息，不是一次完整的数据库、聊天模型和向量服务联合验收。应用启动时还会初始化数据库记忆资源；某些基础服务不可用可能使应用根本无法完成启动。

认证入口统一位于 `/api/v1/access`。客户和企业员工有独立登录身份，即使同一浏览器同时持有两种 Cookie，也会按接口所属身份分别验证。

方法与完整路径	用途
POST <code>/api/v1/access/customer/register</code>	注册客户账号
POST <code>/api/v1/access/staff/register-company</code>	注册企业及相应员工账号
POST <code>/api/v1/access/customer/login</code>	客户登录并设置客户会话 Cookie
POST <code>/api/v1/access/staff/login</code>	员工登录并设置员工会话 Cookie
GET <code>/api/v1/access/customer</code>	查询当前客户身份
GET <code>/api/v1/access/staff</code>	查询当前员工身份及企业归属
POST <code>/api/v1/access/customer/logout</code>	注销当前客户登录会话
POST <code>/api/v1/access/staff/logout</code>	注销当前员工登录会话

客户接口从认证上下文获取客户 ID，不允许通过请求体声明另一位客户来冒用身份。聊天体中的企业和会话标识也要经过归属校验。

方法与完整路径	用途
GET <code>/api/v1/customer/companies</code>	查找可咨询的企业
GET <code>/api/v1/customer/companies/{company_id}</code>	查询企业信息

方法与完整路径	用途
GET /api/v1/customer/conversations	分页查询自己与指定企业的会话
POST /api/v1/agent/chat	提交消息，由 AI 或人工会话状态决定处理方式
GET /api/v1/conversations/{conversation_id}/messages	恢复本人会话消息
GET /api/v1/customer/tickets	分页查询本人可见工单
GET /api/v1/customer/tickets/{ticket_id}/comments	查看客户可见的修改和补充记录
GET /api/v1/customer/companies/{company_id}/orders	查询本人在该企业的模拟订单
POST /api/v1/customer/companies/{company_id}/orders	创建本人模拟订单
GET /api/v1/customer/companies/{company_id}/preferences	读取显式回复偏好
PUT /api/v1/customer/companies/{company_id}/preferences	保存显式回复偏好
DELETE /api/v1/customer/companies/{company_id}/preferences	清除该企业范围内的回复偏好
GET /api/v1/customer/conversations/{id}/handoff	查询本人会话的人工接管状态

客户修改工单通过受控聊天工作流完成，不存在一个允许客户调用企业 PATCH 接口随意修改状态的捷径。客户界面功能是否可见与接口是否存在是两回事，例如偏好 API 不应被误写成已经具备完整设置中心。

方法与完整路径	用途
GET /api/v1/staff/customers	当前企业的客户列表
GET /api/v1/staff/conversations	当前企业会话列表，可按客户筛选
GET /api/v1/staff/conversations/{conversation_id}/messages	员工查看本企业会话
GET /api/v1/staff/handoffs	按待接管或我接管的范围查看队列
GET /api/v1/staff/conversations/{id}/handoff	查询接管状态
POST /api/v1/staff/conversations/{id}/handoff	接管或恢复 AI 处理
POST /api/v1/staff/conversations/{id}/reply	当前接管员工发送人工回复
GET /api/v1/tickets	企业工单列表与筛选
POST /api/v1/tickets	企业人工创建工单

方法与完整路径	用途
GET /api/v1/tickets/statistics	企业工单状态统计
GET /api/v1/tickets/{ticket_id}	工单详情
PATCH /api/v1/tickets/{ticket_id}	企业侧处理工单字段和状态
POST /api/v1/tickets/{ticket_id}/notes	增加企业内部备注
GET /api/v1/tickets/{ticket_id}/messages	查看工单关联会话
GET /api/v1/agent-runs	企业 Agent 运行记录列表
GET /api/v1/agent-runs/statistics	企业运行统计
GET /api/v1/agent-runs/{run_id}	单次运行详情

企业接口的 company_id 来自员工身份。对不属于该企业的资源，即便知道 UUID，也不能合法访问。人工建单时的操作人和客户信息由后端结合身份与会话确定，不信任浏览器伪造的显示名。

知识库接口前缀为 /api/v1/staff/knowledge，只供员工管理自己企业的资料。表中的相对路径应接到此
前缀后。

方法与相对路径	用途
GET 空路径	分页列出资料，当前按 offset 使用服务端页大小
POST 空路径	新建文本资料
POST /upload	上传 TXT 或 Markdown 文本
POST /search	员工测试知识检索
GET /{id}	原文、草稿、发布等详情
POST /{id}/preview	根据原文生成分块草稿
PUT /{id}/draft	保存人工编辑分块，带修订号
POST /{id}/publish	校验并向量化发布
POST /{id}/disable	停用资料，阻止后续正常检索使用
POST /ticket-experiences	从工单信息整理经验草稿
GET /ticket-experiences/candidates	查找可整理经验的工单候选
GET /ticket-experiences/source	按工单编号读取整理所需来源

知识库目前没有通用硬删除接口，也没有通过修改原文自动级联重建全部历史版本的开放协议。发布失败、停用和删除不是同一件事。

后端入口和基础设施

`backend/app/main.py` 组装 FastAPI 应用、路由、中间件、异常处理与生命周期资源。排查“请求根本没到业务层”时先看这里。`core/config.py` 定义环境配置；`core/logging.py` 定义结构化日志；`core/access.py` 提供客户和员工身份依赖及来源校验；`core/passwords.py` 负责密码散列和校验。

`db/base.py` 是 ORM 基础设施，`db/session.py` 管理异步数据库连接及请求会话。`db/models.py` 放会话、消息、工单、运行记录等主体实体；`identity_models.py` 放认证相关实体；`order_models.py` 放模拟订单；`knowledge_models.py` 放知识资料和分块；`chat_operation.py` 定义聊天请求执行回执。它们是数据库映射，不是向大模型发送的提示词。

Repository 层把业务访问范围和 SQL 放在一起。`auth_repository.py` 处理账号、登录会话和限流记录；`conversation_repository.py` 处理会话及消息；`customer_repository.py` 提供客户侧列表；`staff_customer_repository.py` 提供企业客户和会话视图；`order_repository.py` 管理模拟订单与默认订单；`ticket_repository.py` 管理工单主体、备注与写操作；`ticket_query_repository.py` 提供客户安全的工单查询；`agent_run_repository.py` 提供运行记录读写和统计；`chat_operation_repository.py` 提供请求占位、状态与回执查询。`conversation_lock.py` 用 PostgreSQL advisory lock 保护同一会话的并发流程。

数据库外键和唯一约束负责最后一道一致性防护，但业务授权不能只依赖外键。新增 Repository 查询时，必须继承现有企业与客户范围，不能先查全量数据再由前端过滤。

路由与 Schema 文件

`api/routes.py` 放健康检查、聊天和客户历史消息入口；其他业务拆到同名专题路由：`access_routes.py` 管认证，`customer_routes.py` 管客户视图，`order_routes.py` 管模拟订单，`memory_routes.py` 管偏好，`staff_customer_routes.py` 管员工查看客户，`handoff_routes.py` 管人工接管，`ticket_routes.py` 管企业工单，`agent_run_routes.py` 管运行记录，`knowledge_routes.py` 管知识资料和工单经验。

`schema/` 是接口与业务数据契约目录。`access.py` 定义注册、登录和当前身份；`conversation.py` 定义聊天请求、响应和历史消息；`customer.py` 与 `staff_customer.py` 定义两种视角的列表；`order.py` 定义模拟订单输入输出；`handoff.py` 定义接管动作和人工回复；`memory.py` 定义显式偏好；`agent_run.py` 定义运行详情、步骤和统计；`agent_error.py` 定义面向接口的错误结果。

工单契约进一步拆分：`ticket.py` 是企业工单输入输出，`ticket_query.py` 是客户查询结果，`ticket_comment.py` 是客户补充记录，`ticket_edit.py` 是受控字段修改，`ticket_order.py` 是工单关联订单展示，`after_sales.py` 是售后预判结果。`knowledge.py` 定义文档、草稿、片段、检索命中和经验资料。

字段长度、可空、枚举和额外字段处理首先由 Schema 限制。比如聊天文本最多 4000 字符，工单标题通常为 2 至 200 字符，问题描述为 2 至 10000 字符；客户修改中的空字符串与没有提供字段不能随意混同。422 应先核对对应请求 Schema，而不是归咎于模型不可用。

业务服务职责

`auth_service.py` 组织注册、登录和会话逻辑。`conversation_service.py` 是聊天业务的主要协调者，连接认证后的会话、消息保存、Agent 执行、运行记录和结果返回；`chat_request_service.py` 聚焦请求幂等与执行回执，避免重复请求被当作新任务。它们之间的事务边界详见可靠性与误操作处理（第 4 册）。

`agent_memory_service.py` 把数据库业务会话与 LangGraph 状态衔接，处理 thread 配置、状态恢复及异常边界；`customer_memory_service.py` 读写显式长期偏好；`agent_errors.py` 将模型、工具和执行异常归类；`agent_run_service.py` 记录运行成功、失败及摘要；`agent_trace.py` 收集受控执行步骤和安全元数据。

`order_service.py` 校验模拟订单业务条件；`ticket_service.py` 负责建单、企业更新和状态规则；`ticket_query_service.py` 负责客户查询表达与安全结果；`ticket_edit_service.py` 负责客户可编辑字段、版本冲突和修改回执；`ticket_content.py` 将旧的混合描述拆成订单信息、问题与诉求；`comment_receipt.py` 处理客户补充或修改相关回执兼容；`handoff_service.py` 管接管状态、归属和人工回复。

`knowledge_chunking.py` 只聚焦结构化切分，不负责请求向量 API；`knowledge_service.py` 组织草稿、发布、向量校验及检索；`ticket_experience_service.py` 从工单提取可整理经验的来源并创建经验草稿。要调整切分规则应先找前者，要排查维度和发布失败则看知识服务，而不是在提示词里修。

Agent 目录职责

`agent/factory.py` 创建和绑定聊天模型能力，将结构化输出、工具绑定和配置集中管理。`prompts.py` 是模型指令，负责说明任务、边界和输出要求；提示词不能替代服务端权限与数据库约束。`schemas.py` 定义模型结构化判断的数据结构，与公开 API 的 `schema/` 用途不同。

`state.py` 定义图状态；`graph.py` 注册节点和连线，提供确定性路由与模型调用节点；`tools.py` 集中定义可调业务工具，再委托服务层完成数据库操作。修改工具名称会影响绑定模型、提示词、路由、测试和旧检查点兼容，不能只改一个函数名。

`decision_context.py` 整理给意图判断使用的上下文。`workflows/order_workflow.py` 管多轮订单选择和售后建单衔接；`after_sales_workflow.py` 管订单售后预判、依据与确认状态；`ticket_edit_workflow.py` 管选工单、展示可改字段、收集修改及确认；`knowledge_workflow.py` 管检索和基于证据的回答。这些 workflow 是业务编排模块，不是额外启动的后台进程。

`memory/persistence.py` 管官方 PostgreSQL checkpointer、Store、连接池及序列化兼容；`context_window.py` 管发给模型的消息裁剪；`order_memory.py` 管订单选择上下文；`ticket_edit_memory.py` 管工单修改上下文；`comment_memory.py` 管相关补充流程的记忆兼容。

`agent/order_memory.py` 和 `agent/comment_memory.py` 顶层旧路径保留兼容作用。旧检查点可能保存带模块路径的类型，贸然删除兼容入口可能让已有会话无法反序列化。文件看起来重复，不等于可以在没有迁移测试时直接删掉。

节点名称、工具清单和调用顺序集中见聊天链路与 Agent 工作流（第 2 册），本篇不维护第二套图，以免两处不同步。

前端文件导航

frontend/src/router/index.ts 是页面入口真相来源；main.ts 是初始化；App.vue 是应用外壳；styles/main.css 放跨页面基础样式。lib/http.ts 管 Axios，lib/auth-events.ts 管跨标签页认证事件，stores/conversation.ts 管客户临时会话状态。

views/CustomerLayout.vue 是客户身份与页面外壳；CustomerLoginView.vue 和 StaffLoginView.vue 使用 components/AccountAccess.vue；CompanyDirectoryView.vue 是企业选择；CustomerChatView.vue 是当前聊天；CustomerOrdersView.vue 是模拟订单；CustomerTicketsView.vue 是客户工单。企业页面为 StaffCustomersView.vue、KnowledgeView.vue、TicketCenterView.vue 和 AgentRunCenterView.vue。SupportDeskView.vue 与 SupportWorkspace.vue 不应仅凭名字被认作当前路由主入口。

composables/useAgentChat.ts 组织聊天，useChatKeyboard.ts 组织输入键盘行为，useTicketCenter.ts 组织企业工单查询修改，useAgentRunCenter.ts 组织运行记录查询。这些函数利用 Vue 响应式生命周期，不是服务端节点。

services/access-service.ts 管认证请求；agent-service.ts 管聊天；agent-feedback.ts 管错误文案；customer-service.ts 管客户列表接口；staff-customer-service.ts 管企业客户接口；order-service.ts 管订单；ticket-service.ts 管企业工单；handoff-service.ts 管人工状态和回复；knowledge-service.ts 管资料与经验；agent-run-service.ts 管运行记录。types/agent.ts、agent-run.ts、support.ts、ticket.ts、ticket-order.ts 是前端数据形状，不执行数据库操作。

主要复用组件包括 StaffNavigation.vue 企业导航、HandoffInbox.vue 接管队列、HandoffControls.vue 人工操作、TicketCenter.vue 企业工单主体、TicketConversation.vue 关联消息、TicketIssueDetails.vue 统一字段展示、CustomerTicketComments.vue 客户可见记录、KnowledgeDraftEditor.vue 分块编辑、TicketExperienceForm.vue 经验表单和 AgentRunCenter.vue 运行记录。修改一个共享组件时需要同时回归它的所有使用页面。

env.d.ts 提供环境类型；auto-imports.d.ts 和 components.d.ts 配合插件生成类型声明；vite.config.ts 配置 Vue 插件、自动导入和路径别名；tsconfig 系列文件提供编译检查配置；eslint.config.mjs 提供静态规范；playwright.config.ts 配置浏览器测试。生成声明不是业务源码，也不应通过手改声明文件掩盖真实类型错误。

数据库迁移链

迁移脚本位于 backend/alembic/versions/。alembic.ini 配置 Alembic，alembic/env.py 连接数据库并载入模型。将迁移文件放到错误目录会导致 heads 和 history 看不到版本，执行 upgrade 也不会创建预期业务表。

当前迁移依次覆盖以下变化，版本顺序由脚本中的 revision 与 down_revision 决定，不仅由文件名排序决定。

文件名	主要目的
20260906_01_create_conversations.py	会话与消息基础结构

文件名	主要目的
20260911_02_create_tickets.py	工单
20260911_03_add_message_tool_metadata.py	消息工具元数据
20260911_04_create_agent_runs.py	Agent 运行记录
20260911_05_conversation_owner.py	会话归属
20260912_06_dual_accounts.py	双身份认证
20260912_07_scope_integrity.py	租户范围一致性约束
20260914_08_chat_operations.py	聊天请求执行记录
20260915_09_demo_orders.py	模拟订单
20260918_10_ticket_comment_receipt.py	客户补充回执
20260918_11_ticket_edit.py	客户工单修改
20260918_12_separate_ticket_order_content.py	分离订单与问题内容
20260921_13_human_handoff.py	人工接管状态
20260925_14_handoff_queue.py	接管队列相关结构
20260925_15_knowledge.py	知识资料及向量
20260928_16_knowledge_drafts.py	分块草稿与修订
20260929_17_ticket_experience.py	历史工单经验来源
20260930_18_after_sales.py	售后预判所需订单签收信息
20260930_19_agent_steps.py	Agent 执行步骤记录

官方 checkpointer 和 Store 的内部表由相应库的 setup 管理，不能仅查看这 19 个业务脚本就断言数据库没有其他表。数据库迁移也不负责恢复浏览器中的未保存表单。

按现象定位

“消息一直发送中” 先看前端 composable、网络响应和上下文版本，再按请求 ID 查后端，最后才进入模型与 Graph；“选择订单后又问一遍” 先看订单记忆、同一 conversation_id 和图状态恢复；“查不到自己的工单” 先看账号与企业范围、分页、状态筛选，再看查询 Repository。

“知识发布成功但回答没变” 核对当前企业、发布版本、启用状态、Embedding 配置指纹、检索分数和实际命中，不要直接改生成提示词；“历史会话报反序列化错误” 看 persistence 与兼容模块；“按钮不

显示”沿真实路由、身份和状态条件查组件，不要先认定接口不存在。

配置运行测试与维护

本篇面向启动项目、排查问题和接手维护的人。命令用于说明现有操作方式，不代表本次编写文档时执行过数据库迁移、依赖安装或真实模型测试。本知识库的完成依据是源码核对，不能把文档写完当作生产验收通过。

环境与依赖归属

Python 后端依赖安装在仓库根目录的 `.venv`，前端依赖属于 `frontend/node_modules`。激活 Python 虚拟环境不会安装 Node.js、pnpm，也不会把前端包变成 Python 依赖。`requirements.txt` 面向 Python；前端实际依赖以 `frontend/package.json` 和 `pnpm-lock.yaml` 为准。

当前约定是依赖安装由项目所有者执行。开发和文档维护不应顺手下载依赖、浏览器或全局包。缺少命令时应说明缺哪项、在哪个环境，而不是自动替换工具链或向系统环境安装。

前端声明 pnpm 版本并保留锁文件；后端部分关键 LangGraph 相关包固定版本，但不是全部 Python 包都精确锁定。因此现在不能宣称从零重建环境一定逐字节可复现。依赖升级应独立验证，尤其注意检查点序列化、结构化输出、向量客户端和类型检查的兼容性。

配置文件分别放在哪里

后端读取 `backend/.env`，前端本地配置使用 `frontend/.env.local`，两端各有 `.env.example` 作为无真实密钥的模板。后端通过绝对路径定位配置，所以从仓库根目录和 `backend` 目录启动，不应读到两个不同配置文件。

`.local` 表示 Vite 本地环境配置文件的约定，不表示另一种虚拟环境。Vite 的 `VITE_` 变量会进入浏览器可见代码，只能放公开服务地址等信息。聊天模型、Embedding、LangSmith、数据库密码都属于后端机密，不能加一个 `VITE_` 前缀就搬去前端。

当前忽略规则会忽略实际环境文件，但保留示例文件。忽略规则不能撤销过去已经提交或公开的秘密。如果密钥曾出现在聊天、截图或 Git 历史里，应在供应商侧撤销并更换，而不是仅删除文本后继续使用。

配置在后端由 `get_settings()` 缓存。修改环境文件后应重启后端，不应假定每次请求重新读取。Vite 环境配置也需要重启开发服务；生产构建的公开变量通常在构建时进入产物，仅修改部署机器上的文件不会自动改写已有 JavaScript。

后端配置项说明

应用基础项包括 `SUPPORT_APP_NAME`、`SUPPORT_APP_VERSION`、`SUPPORT_DEBUG` 和 `SUPPORT_LOG_LEVEL`。默认应用名为 SupportDesk Agent API，版本为 0.1.0，调试关闭，日志级别 INFO。日志级别变化不应被当作启用全面脱敏的开关。

`SUPPORT_DATABASE_URL` 是 SQLAlchemy 使用的 PostgreSQL 连接地址，当前驱动形式是 `postgresql+psycopg`。主机、端口、库名和账号必须与实际数据库一致；在容器内运行后端时，`127.0.0.1` 指向后端容器自身，不再自动代表宿主机上的数据库。

`SUPPORT_MODEL_NAME` 默认 `deepseek-v4-flash`，`SUPPORT_MODEL_BASE_URL` 默认指向 DeepSeek 地址，`SUPPORT_MODEL_API_KEY` 必须由本地配置提供。这里记录的是当前代码选择，不是对供应商模型生命周期和可用性的外部保证。换模型前应测试工具调用、结构化输出及本项目使用的参数组合，不能只改名字就假设所有能力一致。

`SUPPORT_COOKIE_SECURE` 本地 HTTP 默认 `false`；正式 HTTPS 部署应开启。`SUPPORT_CORS_ORIGINS` 是明确的前端来源列表，默认允许 `localhost:3000` 和 `127.0.0.1:3000`。带 Cookie 的请求需要同时满足浏览器、CORS、Cookie 和后端来源检查规则。开发时统一使用同一种主机写法，可以减少 Cookie 看似登录成功却没有带上的问题。

上下文窗口使用嵌套配置：`SUPPORT_CONTEXT_WINDOW__MAX_MESSAGES` 默认 40，`MAX_TOKENS` 默认 12000，`RESERVE_TOKENS` 默认 2048，`CHARS_PER_TOKEN` 默认 2.0，后三项也使用相同的 `SUPPORT_CONTEXT_WINDOW__` 前缀。这里的条数包括工具消息，不是 40 轮问答；Token 是近似预算，不是模型供应商账单的精确计数。具体裁剪和不能拆散工具轮次的规则见记忆与会话恢复（第 2 册）。

Embedding 与 LangSmith 配置

`SUPPORT_KNOWLEDGE_ENABLED` 默认 `false`。启用知识检索还需要 `SUPPORT_EMBEDDING_API_KEY`、`SUPPORT_EMBEDDING_BASE_URL`、`SUPPORT_EMBEDDING_MODEL` 和 `SUPPORT_EMBEDDING_DIMENSIONS` 一致且有效。默认维度 1536 是配置默认值，不是所有模型的输出维度。

维度允许的配置范围是 1 至 8192，但允许配置不意味着供应商一定能产生该维度。当前调用不能通过随意填写一个数字改变模型输出长度。发布时会校验数量、长度及有限数值，避免将无效向量写入正式索引。

`SUPPORT_KNOWLEDGE_MIN_SCORE` 默认 0.5，允许 -1 至 1。它是余弦相似度阈值，不是 50% 正确率。降低阈值可能增加不相关结果，提高阈值可能漏掉有用资料，应使用真实问题集评估。

更换向量供应商、模型或维度后，应重新发布需要使用的资料。系统用配置指纹隔离不同向量空间，防止旧向量与新查询盲目比较。配置看似正确但一直无命中时，要核对资料发布所用的配置，而不是仅看 `enabled` 为 `true`。

LangSmith 使用标准变量名：`LANGSMITH_TRACING`、`LANGSMITH_API_KEY`、`LANGSMITH_PROJECT`、`LANGSMITH_ENDPOINT` 和可选 `LANGSMITH_WORKSPACE_ID`，不加 `SUPPORT_` 前缀。默认追踪关闭，项目名 `supportdesk-agent-dev`。开启前应确认服务端数据可能被发送到该外部追踪服务，尤其是输入消息、提示词和工具链上下文；业务日志的精简字段策略不等于 LangSmith 自动遵循相同策略。

启动脚本如何工作

从项目根目录执行 `bash scripts/start.sh`。脚本根据自身文件所在位置找到仓库根目录，不根据终端当前目录猜测 `.venv`。因此脚本位于 `scripts` 文件夹时，应检查其父目录定位逻辑，而不是为了绕过错误再创建一套虚拟环境。

启动器先核对项目 Python、必要模块、Docker、Node、配置文件、现有前端依赖以及端口。`bash scripts/start.sh --check` 只检查预备条件，不启动完整服务，也不证明 Docker 守护进程、模型或向量接口一定能完成业务请求。

正式启动会检查 Docker。macOS 上在必要时尝试打开 Docker Desktop，并等待其就绪；其他环境需要事先准备可用的 Docker 服务。仅能运行 `docker` 命令不代表 Docker daemon 已启动，这正是 `Cannot connect to the Docker daemon` 的常见原因。

随后启动 PostgreSQL 并等待健康检查，再执行 Alembic 数据库迁移。迁移失败会阻止继续启动前后端，避免页面已经出现而业务表尚未准备好的半启动状态。迁移会修改数据库结构，上线前必须单独评估并备份。

后端以开发模式 Uvicorn 启动，监听 127.0.0.1:8000；前端通过已安装的 Vite 启动，监听 127.0.0.1:3000，使用严格端口模式，不会静默换到 3001。启动器等待后端健康地址和前端 HTTP 响应，再输出各服务结果。

正常访问地址是前端 <http://127.0.0.1:3000/>、接口文档 <http://127.0.0.1:8000/docs> 和健康检查 <http://127.0.0.1:8000/health>。Docker 的成功提示特指数据库容器就绪，不表示所有未来业务查询都永不失败，也不表示真实模型配置已经验证。

日志分别追加到 `logs/backend.log` 和 `logs/frontend.log`。当前没有自动日志轮转，长期使用需要控制磁盘增长。启动器运行期间监视子进程；按 `Ctrl+C` 会停止本次启动的前后端进程组，PostgreSQL 保持运行。它不会为了启动新服务而自动杀死原本占用端口的未知进程。

数据库持久化和迁移排查

Compose 使用 `pgvector/pgvector:pg17`，数据库容器为 `supportdesk-postgres`，命名卷为 `supportdesk_postgres_data`。停止容器或重启 Docker 通常保留卷中数据；删除卷、清理 Docker 数据或损坏磁盘则可能丢失业务、记忆和知识索引。

持久化卷不是备份。正式数据应有独立备份、恢复演练和保留策略，不能把“重启后还在”当作“任何误操作都能恢复”。当前项目没有自动备份和一键恢复控制台。

迁移命令应在 `backend` 目录、使用项目 Python 环境执行。`alembic heads` 查看源码最新版本，`alembic history` 查看迁移链，`alembic current` 查看目标数据库当前版本，`alembic upgrade head` 才执行升级。只看命令没有报错，不足以确认预期版本存在。

若数据库只有 `alembic_version`，没有会话和工单表，优先核对迁移目录、版本文件内容、数据库连接目标和当前版本。不要直接把版本号手工标成最新以掩盖表缺失；这会让后续升级误以为结构已准备好。

应用业务数据和官方记忆资源使用不同连接与事务。业务数据库恢复到昨天、检查点却停在今天，可能造成会话指向不存在的业务回执。因此备份和恢复应一起考虑业务表、官方记忆表、知识原文、草稿和向量索引，不能只导出工单表。

日志和运行记录怎样联合使用

每个 HTTP 请求会关联请求 ID。客户端传来的合法标识可以被采用，不合法或缺失时会生成新标识。完成日志包含方法、路由、状态码、耗时和请求 ID；响应头也提供排查线索。

业务运行记录回答的是“哪次 Agent 执行、属于哪个会话、用了什么工具、成功还是失败”；HTTP 日志回答的是“哪次网络请求、用了多久、返回什么”。一条聊天消息可能涉及多次内部模型调用，两者不能直接按数量一一等同。

运行步骤追踪最多保留受限数量的记录，记录安全白名单单元数据，不是任意参数全文。已有旧记录可能没有步骤，当前步骤也不是所有内部函数的完整调用栈。平均耗时统计以成功运行为范围，工具统计口径见前端页面与交互实现（第 4 册）。

异常堆栈可能包含库产生的细节，所以日志仍应限制访问、定期清理，分享前检查敏感数据。不要为查 502 临时打印整个 settings、Authorization、Cookie 或模型密钥。

常见故障的处理顺序

端口占用时，先确认是不是之前启动的前端或 Uvicorn reload 子进程仍在，再从原终端停止。不要用宽泛进程匹配批量杀死 Python 或 Node。端口检测基于实际监听和可绑定性，终端一次没列出进程不必然证明端口从未被短暂占用。

401 优先检查登录身份、Cookie 是否带上、会话是否过期；403 查看来源或权限条件；404 同时可能表示不存在或无权查看，不能通过修改请求中的客户 ID 绕过。409 通常要看并发、版本或请求状态，不应直接无限重试。

422 表示请求契约未通过。查看后端返回的字段位置，核对枚举、长度、UUID、金额、日期和空字符串。前端没有红框不代表服务端必须接受。修改校验时要保持前端提示与后端真实规则一致。

502 在本项目常见于模型、结构化解析、工具或上游链路异常，但必须根据错误码和日志进一步区分。先保存请求 ID，再核对运行记录、业务回执和工单列表，最后检查模型地址、参数、限流或响应格式。不能把每个 502 都改成“重试一次”处理。

前端 Axios 默认等待 60 秒；模型客户端和 Graph 分支另有各自的超时与调用预算，查询相关分支状态目前设置 90 秒 deadline。这些不是一个统一的端到端 60 秒事务。因此浏览器超时后服务端可能还在执行，应该先核对结果。

“上一轮尚未完成”意味着检查点或业务回执需要恢复判断，并不证明上轮没有写入。当前只对明确安全的部分失败状态做自动收尾，不会无条件恢复工具写入。管理员应对照请求回执、工单和运行状态处理，避免直接删检查点或重放写工具。

知识服务未启用时，检查 enabled 和独立向量配置；向量维度错误时，核对实际模型输出；无命中时，检查企业、资料状态、索引配置指纹和分数；回答错误时，进一步区分没有召回、召回不相关和生成偏离证据。三种问题不能全靠加长提示词解决。

自动化测试分层

后端测试位于 backend/tests/，覆盖认证、接口契约、Graph 分支、上下文裁剪、订单记忆、模拟订单、工单查询及修改、人工接管、知识切分及发布、历史经验、检查点恢复、售后预判和启动端口检查等。测试替身可以验证控制流，不等于真实 DeepSeek 每次都产生相同答案。

部分测试需要 PostgreSQL，并通过 SUPPORT_TEST_DATABASE=1 显式启用。执行前阅读测试夹具并确认连接的是可用于测试的数据库，不要将带写入或清理逻辑的测试指向唯一生产库。仅看到若干单元测试通

过，不能认为这些被跳过的集成测试也已通过。

前端通过 vue-tsc 检查类型、ESLint 检查规范、Vite build 验证构建，Playwright 检查界面行为。浏览器测试使用 3101 独立预览端口和模拟接口，不复用正在运行的业务服务，也不直接证明真实后端授权或真实模型成功。

Playwright 需要先有构建产物，并有可用浏览器。配置允许指定本机已安装 Chrome，使用隔离测试配置而非个人浏览资料。缺浏览器或依赖时应报告前置条件，不在未经安排的情况下自动下载。

可在已经准备好依赖的环境中使用以下检查顺序。命令只运行已有工具，不包含安装步骤；数据库测试是否开启应由操作者确认。

```
# 在项目根目录进入后端；这些检查不会代替真实模型验收。
cd backend
../.venv/bin/ruff check app tests
../.venv/bin/mypy app
../.venv/bin/pytest

# 前后端是同级目录，因此从 backend 进入 frontend 要先回上一层。
cd ../frontend
pnpm typecheck
pnpm lint
pnpm build
```

测试报告应注明提交或工作区版本、执行命令、环境、通过与跳过数量、是否使用真实外部服务。当前知识库不把此前某一轮的测试数量当作本次重新验证结果，也不承诺今后新增测试后数量不变。

人工验收场景

账号与隔离验收需要至少两个客户、两个企业及多个员工身份。客户甲应看不到客户乙的订单和会话；企业甲应看不到企业乙的工单、知识和运行记录；复制 UUID 不应突破范围。测试数据应明确标记为模拟，避免误发给真实客户。

订单建单验收应覆盖“列出订单一回复序号一只回复退款或维修一确认结果”，检查跨轮次是否保留所选订单；换订单后原确认应失效。售后预判应覆盖日期缺失、不符合条件、资料不足、客户明确确认，以及规则或订单信息改变后重新评估。

工单修改验收应覆盖只有一条也先选择、只改一个字段其他保持不变、企图改状态被拒绝、编辑期间员工更新导致版本冲突。结果应与客户工单页、企业详情和修改记录一致。

RAG 验收可使用独特且明确标注模拟的规则和验证口令，分别问直接事实、上下文追问、其他商品是否适用以及文档没有覆盖的问题。验证不仅看能否复述，还要看能否拒绝把规则扩展到无关商品。测试资料结束后应停用，避免混入正常售后回答。

人工接管验收应从客户申请开始，在企业一级页面发现待接管记录，员工接管并回复，客户通过轮询看到回复，最后交回 AI。还应测试两位员工同时接管及当前接管者退出页面，确认实际行为与提示一致。

误操作验收应覆盖请求中刷新、关闭页面、重复点击、新开标签页、登出后登录另一个账号、网络响应丢失、知识草稿版本冲突、服务器重启。对于目前不保留的草稿，应验证提示与边界，而不是以“以后会完善”为理由报告已通过恢复测试。

生产化尚缺的工作

目前启动器是本地开发工具，不是生产进程管理平台。生产化仍需要独立考虑 HTTPS、反向代理、受控 CORS、Cookie 策略、静态资源与 SPA 回退、数据库暴露范围、密钥管理、备份恢复、日志轮转、资源限制、告警、升级回滚及长连接池容量。

业务上仍缺完整员工管理与角色细分、真实订单及支付系统、工单 SLA 自动升级、员工在线与转派机制，以及更系统的模型质量评估。知识图谱没有实现；依赖清单中出现 Redis、Celery 或检索库，不表示已经有缓存、任务队列或混合检索服务。

记忆和运行数据没有一套完整的保留期限与用户删除工作流。需要删除某位客户数据时，要同时考虑业务会话、检查点、长期 Store、请求回执、运行记录、知识案例中的信息和外部追踪平台，不能只删 messages 就宣布完成隐私删除。

未来每次改动应先明确业务不变量和恢复策略，再更新 Schema、服务、工具、图状态、前端和测试中真正受影响的部分。涉及类型路径、线程 ID、请求幂等或权限范围的修改，需要把旧数据兼容性列为验收项。相关文档同步更新后，才能作为下一位开发者或 AI 的可靠项目上下文。